



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

ASP.NET CORE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on pipeline, DI lifetimes, Minimal API, auth, and testing

TOTAL: 10 QUESTIONS

Q1 What is the ASP.NET Core request pipeline and how do you add middleware? dry model

Q6 How do you add JWT Bearer authentication to ASP.NET Core? JWT

Q2 What are the three DI service lifetimes and what are their dependency risks? capture

Q7 What is the Options pattern in ASP.NET Core configuration? cycle

Q3 How does ASP.NET Core attribute routing work? Configuration

Q8 What is Entity Framework Core and how does Add-Migration work? migration

Q4 What is the difference between [FromBody], [FromQuery], and [FromRoute]? body

Q9 What are action filters in ASP.NET Core and what can they do? do

Q5 What is Minimal API in ASP.NET Core 6+ and when should you use it? use

Q10 How do you write integration tests for ASP.NET Core? Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 The pipeline is a chain of middleware

Mitigation

The pipeline is a chain of middleware delegates in Program.cs. `app.UseRouting()`, `app.UseAuthentication()`, and `app.UseAuthorization()` must be in that order. `app.Use(async (ctx, next) => {...; await next(ctx)})` inserts custom middleware.

A6 Call `builder.Services.AddAuthentication().AddJwtBearer(opts)`

Security

Call `builder.Services.AddAuthentication().AddJwtBearer(opts => {opts.TokenValidationParameters = new() {ValidIssuer=..., IssuerSigningKey=...}})`. Add `app.UseAuthentication()` and `app.UseAuthorization()` in the pipeline. Protect endpoints with `[Authorize]`.

A2 Singleton lives for the app lifetime. Scoped

Primitives

Singleton lives for the app lifetime. Scoped lives per HTTP request. Transient is created fresh each injection. A captive dependency occurs when a Singleton injects a Scoped service the Scoped service is never disposed correctly. ASP.NET Core detects this at startup.

A7 Define a class `MySettings` with properties. Call

Automation

Define a class `MySettings` with properties. Call `builder.Services.Configure<MySettings>(builder.Configuration.GetSection("MySettings"))`. Inject `IOptions<MySettings>` (singleton snapshot), `IOptionsSnapshot<MySettings>` (scoped, reloads per request), or `IOptionsMonitor<MySettings>` (live).

A3 `[Route("api/users")]` on a controller and `[HttpGet("{id:int})]` on

Routing

`[Route("api/users")]` on a controller and `[HttpGet("{id:int})]` on an action define the full route template. Route constraints like `:int`, `:guid`, and `:minlength(3)` validate segments. `[ApiController]` enforces model binding and returns 400 on validation failure.

A8 EF Core is Microsoft's ORM. `DbContext` represents

Governance

EF Core is Microsoft's ORM. `DbContext` represents a session with the database. Define `DbSet<T>` properties for each entity. `dotnet ef migrations add AddUserTable` generates a migration C# file based on the difference between the model and the last migration.

A4 `[FromBody]` reads from the JSON request body

Gotchas

`[FromBody]` reads from the JSON request body (only one per action). `[FromQuery]` reads query string parameters. `[FromRoute]` reads named route template segments. Without explicit attributes, `[ApiController]` infers: complex types from body, simple types from query/route.

A9 `IActionFilter` runs `OnActionExecuting` (before the action) and

Logging

`IActionFilter` runs `OnActionExecuting` (before the action) and `OnActionExecuted` (after). Use them for logging, validation, or caching. `[ResponseCache]`, `[ValidateAntiForgeryToken]` are built-in filters. Register globally in `AddControllers(opts => opts.Filters.Add(...))`.

A5 Minimal API uses lambda-based route handlers

Federation

Minimal API uses lambda-based route handlers (`app.MapGet("/users/{id}", (int id) => ...)`) instead of controllers. It is less structured but has lower overhead and works well for microservices or simple CRUD APIs where the MVC controller ceremony is unnecessary.

A10 Use `WebApplicationFactory<Program>` to spin up the app

Testing

Use `WebApplicationFactory<Program>` to spin up the app in-process. Call `factory.CreateClient()` to get an `HttpClient` that sends requests to the test server. Override services with `factory.WithWebHostBuilder(b => b.ConfigureTestServices(...))`. No real port is needed.